

Báo cáo Big Data Lab 3 — MapReduce và Spark

Trạng thái evidence: phần phương pháp và kiểm thử local đã hoàn tất. Các ô **CẦN ĐIỀN SAU KHI CHẠY LAB 1** phải được thay bằng log/counter/timing thật do `scripts/run-all.sh` sinh ra; tuyệt đối không nộp số liệu giả.

1. Môi trường và dữ liệu

- Ngôn ngữ: Scala 2.11.12; Java 8; Hadoop 3.3.6; Spark baseline 2.4.8/Scala 2.11.
- Dataset: 128.975 rows, khoảng 68,9 MB; grain là một CSV row và `index` là `record_id`.
- Khoảng ngày đã profile: 2022-03-31 đến 2022-06-29; 7.795 Amount null.
- CSV được đọc bằng schema tường minh/Commons CSV; không dùng `split(",")` cho record và không infer schema Spark.

2. Task 1-1 — Dynamic sliding window

2.1 Phân rã truy vấn

Bought row thỏa `Status` sau chuẩn hóa có chứa `SHIPPED` và `Qty != 0`. Job A đếm bought rows theo state để chọn `w=5` nếu tổng `>10000`, ngược lại `w=10`. Job B thực hiện map-to-buckets: row ngày `t` phát đúng `w` khóa cho `t+1..t+w`. Như vậy bucket ngày `d` chứa đúng interval `[d-w, d-1]`, kể cả timestamp chưa xuất hiện trong input. Job C chọn size thắng theo frequency giảm dần, population variance Amount tăng dần, rồi size lexicographic tăng dần.

2.2 Tính đúng và độ phức tạp

Accumulator giữ `count`, `amountCount`, `sum`, `sumSquares`; phép combine có tính kết hợp nên dùng combiner an toàn. Amount null vẫn tăng frequency nhưng không vào variance. Một Amount cho variance 0; không có Amount cho variance undefined và thua variance hữu hạn khi frequency hòa.

Mỗi bought row phát tối đa 10 record nên chi phí map là $O(n*w)=O(n)$ với $w \leq 10$, tốt hơn quét lại toàn input cho mỗi state/ngày ($O(D*n)$). Combiner giảm raw map outputs xuống số bucket-size cục bộ.

2.3 Evidence cần chèn từ Lab 1

- Job A/B/C wall time: **CẦN ĐIỀN SAU KHI CHẠY LAB 1.**
- `MAP_OUTPUT_RECORDS`, `COMBINE_OUTPUT_RECORDS`, `REDUCE_SHUFFLE_BYTES`: **CẦN ĐIỀN.**
- Số dòng output và checksum `Task_1-1.csv`: **CẦN ĐIỀN.**

Unit tests đã phủ ngưỡng 10.000/10.001, bucket tương lai, frequency/variance/lexical ties, null Amount và variance clamp.

3. Task 1-2 — Median variety

“Purchased goods” được hiểu là bought rows theo cùng predicate Task 1-1; cancelled/Qty 0 không phải hàng đã mua. Job A group (state, month, style), đếm distinct normalized SKU và OR điều kiện có size $\geq$ XXL (XXL, 2XL, XXXL, 3XL, ...). Chỉ style qualifying được phát sang Job B. Job B sort các variety theo (state, month) và tính exact median: odd lấy giữa, even lấy trung bình hai giữa.

Độ phức tạp median của k styles trong một state-month là $O(k \log k)$. Tests đã phủ duplicate SKU, alias size, month boundary, odd/even/empty và loại non-bought.

- Counters/timing hai job: **CẦN ĐIỀN SAU KHI CHẠY LAB 1.**
- Số dòng/checksum Task_1-2.csv: **CẦN ĐIỀN.**

4. Task 2-1 — Cancelled Standard percentage

4.1 DataFrame pipeline

Promotion được explode, trim, bỏ rỗng và distinct theo record; không loại promotion Amazon. Một promotion hợp lệ khi `datediff(max(date), min(date)) >= 2` trên toàn dataset. Trung bình Amount theo state chỉ dùng exact normalized `FULFILMENT=MERCHANT`, `COURIER_STATUS=SHIPPED`, Amount hợp lệ.

Mẫu số là toàn bộ Cancelled + Standard rows theo (state, city). Tử số thêm: ít nhất 3 promotion hợp lệ, Amount non-null và Amount nhỏ hơn state average. City thiếu average vẫn được giữ với tử số 0. Percentage là $100.0 * \text{numerator} / \text{denominator}$.

4.2 Plan, join, Exchange và stage

Pipeline chỉ dùng Structured API; không gọi trực tiếp `spark.sql`. `extended-plan.txt` chứa parsed/analyzed/optimized/physical plan từ `explain(true)`. `execution-summary.txt` ghi join operator thực tế, số node Exchange và unique stage IDs của job group.

- Join strategy thật: **CẦN CHÉP** từ `docs/evidence/task21/execution-summary.txt`.
- Exchange count: **CẦN ĐIỀN.**
- Stage count/IDs: **CẦN ĐIỀN.**

Fixture local[2] tạo plan có join/Exchange và cho kết quả tính tay: STATE A/CITY SAME = $1/2 = 50\%$, CITY ZERO = 0%, cùng tên city ở STATE B không bị trộn.

5. Task 2-2 — Approximate và exact percentiles

5.1 Hai phương pháp

Population là mọi row có SKU/date hợp lệ; `promotion_count` là số token distinct trong row, kể cả Amount null. Approx dùng built-in `percentile_approx` với accuracy mặc định 10.000 qua một DataFrame `expr` được duyệt. Exact dùng Window order (`promotion_count, record_id`) và nearest-rank `r=ceil(p*N)` cho P80/P90; threshold là observed promotion count tại rank đó.

Mỗi method/level giữ rows có `promotion_count >= threshold` . Output ghi qualifying row count, non-null Amount count và `stddev_pop` . Nếu qualifying count <2 hoặc không có Amount thì SD=0.

5.2 So sánh độ chính xác

`threshold-deltas` ghi `approx-exact` theo SKU/month/level. Hai left-anti join theo `record_id` đo `approx_only` và `exact_only` ; evidence chỉ giữ tối đa 100 examples. Tests nearest-rank đã pass cho N=1/2/5/10, duplicate threshold và equality boundary.

- Tổng group/threshold khác nhau và symmetric differences: **CẦN ĐIỀN TỪ EVIDENCE**.

5.3 Benchmark công bằng

Cùng một SKU-month base được cache/materialize trước hai method. Mỗi method warm-up một lần; 5 measured runs được xen kẽ thứ tự để giảm bias nhiệt/GC và đều ép full `foreachPartition` action. Báo mean số học và sample SD với mẫu số `n-1` .

Method	Run count	Mean ms	Sample SD ms
approx	5	CẦN ĐIỀN	CẦN ĐIỀN
exact	5	CẦN ĐIỀN	CẦN ĐIỀN

5.4 Group lớn và partition

Profiling ban đầu cho thấy max SKU-month là 426 rows và không có group >1.000. So với input khoảng 68,9 MB (dưới mốc 128 MB), từng group nhỏ; không có lợi ích rõ ràng khi manual repartition theo group. Catalyst/shuffle partitioning được giữ, và `coalesce(1)` chỉ dùng sau aggregate để commit file output nhỏ.

- `valid_rows` , group count, max/group >1.000 từ `group-profile.txt` : **CẦN XÁC NHẬN TRÊN LAB 1**.

6. Kiểm thử, output và khả năng tái lập

- Pure/unit tests: parser, normalization, window, moments/ties, exact median, benchmark math.
- Spark local[2]: promotion, metrics, query results, percentile boundaries, plan/Exchange/stages.
- Target E2E: chạy `scripts/run-all.sh` , sau đó `scripts/validate-outputs.sh` đọc lại bốn exact physical files.
- Output bắt buộc: `Task_1-1.csv` , `Task_1-2.csv` , `Task_2-1.parquet` , `Task_2-2.parquet` .
- `shapes.parquet(legacy)` không liên quan và không nằm trong submission.

7. Kết luận

Thiết kế MapReduce tránh repeated scans bằng bounded bucket emission và combiner; thiết kế Spark giữ row grain, schema tường minh và evidence của optimizer/runtime. Các giả định còn mơ hồ của đề (bought rows cho Task 1-2, mẫu số Task 2-1, nearest-rank exact) đều được nêu công khai và kiểm thử.

Báo cáo chỉ hoàn tất để nộp sau khi thay toàn bộ marker evidence bằng số liệu thật từ môi trường Lab 1.